

Vision Day01 hw1 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.21

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 과제에서 ROS2 를 이용하여 하나의 패키지 안에 카메라 노드와 UI 노드를 구성하는 과정에서 ChatGPT 를 활용하였다. AI 는 과제의 구조와 코드가 동작하는 원리를 이해하고, 필요한 파일을 구성하는데 사용하였다. 먼저 구조를 CameraNode → Topic → UINode 형태로 정리한 뒤, ROS2 패키지를 생성하고 amera-node, ui_node, config, launch 폴더를 구성하였다. 과제의 구현 조건에 맞춰서 AI 와의 대화를 하며 코드를 구성하였다.

2. AI 사용 부분

처음에는 ROS2 패키지 구조를 구성하는 과정에서 AI 를 사용하였다. 패키지를 직접 만들면서 처음에는 단순히 폴더만 생성하여 CMakeLists.txt 와 package.xml 이 없는 상태가 되었는데, AI 에게 현재 tree 결과를 보여주고 문제점을 확인하였다. 이후 ros2 pkg create 명령어를 사용해야 정상적인 ROS2 패키지가 생성된다는 것을 확인하고 다시 패키지를 구성하였다

○ㅋ, 이제 보니까 위치는 제대로 ~/colcon_ws/src/camera_ui_pkg 로 잡혔는데 한 가지 문제가 있음.

지금 tree 결과 맨 아래가

```
</> 일반 텍스트
10 directories, 1 file
```

이고, 파일이 mainwindow.h 하나뿐이잖아. 즉 CMakeLists.txt 랑 package.xml 이 없음. 지금은 폴더만 만든 상태라서 아직 ROS2 패키지가 아니야.

그리고 mainwindow.h 는 이번 PPT 과제 구조에는 안 나오는 파일이라 일단 필요 없음. PPT에서는 camera_node.hpp , ui_node.hpp 를 요구하고 있어.

안전하게 지금 폴더를 백업해놓고 제대로 패키지를 다시 만들자.

```
</> Bash
cd ~/colcon_ws/src

mv camera_ui_pkg camera_ui_pkg_backup

ros2 pkg create camera_ui_pkg --build-type ament_cmake
```

그다음 들어가서 필요한 폴더 생성:

```
</> Bash
cd camera_ui_pkg

mkdir -p src/camera_node
mkdir -p src/ui_node

mkdir -p include/camera_ui_pkg/camera_node
mkdir -p include/camera_ui_pkg/ui_node

mkdir -p launch
mkdir -p config
```



코드 작성 과정에서도 전반적으로 AI의 도움을 받았다. 카메라 노드에서는 OpenCV의 VideoCapture를 이용해 카메라 영상을 받아오고, cv_bridge를 이용하여 cv::Mat 형태의 이미지를 ROS2의 sensor_msgs::msg::Image 형태로 변환한 뒤 Topic으로 publish하도록 작성하였다. UI 노드에서는 해당 Topic을 subscribe하여 이미지를 받아오고 다시 OpenCV 이미지로 변환한 후 화면에 출력하도록 구성하였다.

또한 과제 조건에 있는 mutex를 사용하는 부분에서도 AI의 도움을 받았다. UI 노드에서는 image callback에서 새로운 이미지를 저장하고 timer callback에서는 저장된 이미지를 읽어서 화면에 출력한다. 이때 두 callback이 같은 이미지 데이터에 동시에 접근할 수 있기 때문에 std::mutex와 std::lock_guard를 이용하여 latest_image_를 보호하도록 코드를 작성하였다. 단순히 과제 조건에 맞추기 위해 mutex를 사용하는 것이 아니라, 공유 데이터에 동시에 접근하면서 발생할 수 있는 문제를 막기 위한 것이라는 설명도 함께 들으며 코드를 작성하였다.

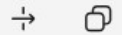
응, ui_node.hpp까지는 잘 들어갔어. 이제 다음은 src/ui_node/ui_node.cpp 만들면 됨.

왼쪽에서 src/ui_node 안에 ui_node.cpp 만들고 아래 코드 넣어.

```
</> C++  
  
#include "camera_ui_pkg/ui_node/ui_node.hpp"  
  
#include <cv_bridge/cv_bridge.hpp>  
  
#include <chrono>  
#include <functional>  
  
UiNode::UiNode()  
: Node("ui_node")  
{  
    this->declare_parameter<double>("display_hz", 30.0);  
    this->declare_parameter<std::string>("image_topic", "/camera/image");  
  
    display_hz_ = this->get_parameter("display_hz").as_double();  
    image_topic_ = this->get_parameter("image_topic").as_string();  
  
    image_sub_ =  
        this->create_subscription<sensor_msgs::msg::Image>(  
            image_topic_,  
            10,  
            std::bind(  
                &UiNode::imageCallback,  
                this,  
                std::placeholders::_1));  
  
    auto period =  
        std::chrono::duration<double>(1.0 / display_hz_);  
  
    timer_ = this->create_wall_timer(  
        std::chrono::duration_cast<std::chrono::milliseconds>(period),  
        std::bind(&UiNode::timerCallback, this));  
  
    cv::namedWindow("Camera", cv::WINDOW_AUTOSIZE);  
}  
  
void UiNode::imageCallback(  
    const sensor_msgs::msg::Image::SharedPtr msg)  
{  
    cv::Mat image;
```

두 번째가 이번 과제에서 중요한 `mutex`.

</> C++



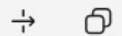
```
std::lock_guard<std::mutex> lock(image_mutex_);

latest_image_ = image.clone();
```

이게 아까 교수님이 말한 `***Image는 mutex로 보호***`를 실제로 구현한 부분이야.

`lock_guard`가 생성되는 순간:

</> 일반 텍스트



`image_mutex_` 잠금

되고, 함수나 `{ }` 범위를 빠져나가면 자동으로:

</> 일반 텍스트



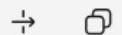
`image_mutex_` 해제

됨.

그래서 `imageCallback()`이 이미지를 저장하는 동안 `timerCallback()`이 같은 이미지에 접근하는 걸 막는 거야.

그리고 `timerCallback()`에는 일부러 이렇게 `{ }`가 하나 더 있어.

</> C++



```
{
    std::lock_guard<std::mutex> lock(image_mutex_);

    if (latest_image_.empty())
    {
        return;
    }

    image = latest_image_.clone();
}
```



3. 깨달은 점

가장 새롭게 배운 부분은 mutex 였다. 이전까지는 프로그램이 위에서 아래로 한 줄씩 실행된다고만 생각했기 때문에 하나의 변수에 여러 callback 이 동시에 접근할 수 있다는 상황을 크게 생각하지 않았다. 그러나 ROS2 에서는 callback 과 timer 등이 각각 실행되면서 같은 데이터를 사용할 수 있기 때문에 공유되는 데이터를 안전하게 관리하는 것이 중요하다는 것을 알게 되었다. 이번 과제를 통해 `std::mutex` 와 `std::lock_guard` 가 이러한 상황에서 데이터 접근을 제어하는 역할을 한다는 것을 깨달았다.

앞으로 비슷한 ROS2 과제를 진행할 때는 먼저 노드가 각각 어떤 역할을 하는지 정리하고, 어떤 Topic 을 이용해 데이터를 주고받는지 설계한 뒤 코드를 작성하는 방식으로 진행하는 것이 좋다고 생각하였다. 또한 AI 가 제시한 코드를 그대로 사용하는 것이 아니라 각 함수와 변수의 역할을 확인하고 직접 실행 및 수정하면서 이해하는 과정이 필요하다고 느꼈다.